

REKAYASA PERANGKAT LUNAK

Modul Pembelajaran: Pemodelan Perangkat Lunak dan UML

Penjelasan komprehensif mengenai konsep representasi sistem, landasan metodologi berorientasi objek, serta standar visualisasi diagram UML.

Contents

1	Pemahaman Dasar Pemodelan Sistem	2
1.1	Karakteristik Kualitas Proses Pemodelan Analisis	2
2	Landskap Metodologi Analisis	3
3	Pengenalan Unified Modeling Language (UML)	4
4	1. Use Case Diagram	5
5	2. Activity Diagram	6
5.1	Simbol-Simbol Activity Diagram	6
6	3. Sequence Diagram	7
6.1	Simbol-Simbol Sequence Diagram	7

1 Pemahaman Dasar Pemodelan Sistem

Sebelum menulis baris kode program, seorang *Software Engineer* harus mampu merancang cetak biru (*blueprint*) dari sistem yang akan dibangun. Proses perancangan ini disebut dengan **pemodelan**.

Secara definisi, **Pemodelan (Model)** adalah representasi dari sebuah sistem nyata yang telah disederhanakan. Tujuannya agar kompleksitas sistem dapat lebih mudah dipelajari, dipahami, dan dikomunikasikan antar anggota tim. Dengan menggunakan model, tim pengembang dapat melakukan visualisasi, spesifikasi, konstruksi, dan dokumentasi secara lebih terstruktur sebelum sistem diimplementasikan dalam bentuk kode.

Ada 5 macam gaya atau cara merepresentasikan sebuah model sistem:

1. **Physical (Fisik):** Membuat versi nyata dengan skala yang diperkecil (*scaled-down version*) dari sistem yang dipelajari. Contohnya: miniatur model pesawat, model kereta api, atau prototipe hardware yang mendemonstrasikan aspek fungsional dari sistem sebenarnya.
2. **Pictorial (Gambar):** Representasi menggunakan lukisan atau gambar untuk mendeskripsikan suatu bidang atau fenomena. Contohnya: peta topografi kontur bumi, bola dunia (globe), atau diagram aliran air dalam sistem hidrologi.
3. **Verbal:** Representasi suatu sistem menggunakan susunan kata dan kalimat verbal yang mendeskripsikan ukuran, bentuk, dan karakteristik sistem tersebut. Pendekatan ini bermanfaat untuk dokumentasi requirements dan spesifikasi kebutuhan yang ditulis dalam bentuk narasi atau daftar kebutuhan.
4. **Schematic (Skema):** Representasi dalam bentuk skema figuratif yang menggabungkan simbol-simbol dengan garis penghubung. Contohnya: skema rangkaian listrik, skema alur proses produksi, atau model atom Bohr dalam kimia.
5. **Symbolic (Simbolik):** Representasi ke dalam bentuk simbol-simbol matematika, di mana karakteristik variabel diformulasikan menggunakan persamaan matematika. Pendekatan ini sangat berguna untuk sistem yang memiliki perilaku kuantitatif atau algoritma kompleks yang dapat diekspresikan secara matematis.

1.1 Karakteristik Kualitas Proses Pemodelan Analisis

Saat melakukan proses pemodelan analisis, kita harus mengevaluasi apakah proses yang kita jalankan sudah baik atau belum. Evaluasi ini penting untuk memastikan bahwa model yang dihasilkan dapat digunakan secara efektif oleh seluruh tim pengembang. Ada 8 indikator kualitas proses analisis yang perlu dipertimbangkan:

- **Understandability (Kemudahan Pemahaman):** Sejauh mana proses didefinisikan secara eksplisit dan seberapa mudah definisi proses itu dipahami oleh anggota tim. Model yang baik harus dapat dipahami oleh semua stakeholder tanpa memerlukan penjelasan yang berbelit-belit.
- **Visibility (Visibilitas):** Apakah aktivitas proses menghasilkan titik akhir (keluaran) yang jelas, sehingga kemajuan dari proses tersebut terlihat nyata. Setiap tahap harus menghasilkan deliverable yang terukur dan dapat diverifikasi.

- **Supportability (Dukungan Alat):** Sejauh mana aktivitas proses dapat didukung oleh alat bantu CASE (*Computer-Aided Software Engineering*). Alat seperti ArchiMate, Enterprise Architect, atau Lucidchart dapat mempercepat proses pemodelan.
- **Acceptability (Penerimaan):** Apakah proses yang telah ditentukan dapat diterima, digunakan, dan dipertanggungjawabkan selama pembuatan perangkat lunak. Proses harus selaras dengan budaya organisasi dan tidak menimbulkan resistensi dari tim.
- **Reliability (Keandalan):** Apakah proses didesain sedemikian rupa sehingga *error* (kesalahan) proses dapat dihindari sebelum merusak produk akhir. Mekanisme quality assurance dan review harus tertanam dalam proses.
- **Robustness (Ketangguhan):** Dapatkah proses pengembangan terus berjalan walaupun terjadi masalah *error* yang tidak diduga di tengah jalan. Proses yang robust memiliki mekanisme fallback dan recovery yang baik.
- **Maintainability (Kemudahan Pemeliharaan):** Dapatkah proses berkembang dan beradaptasi untuk mengikuti perubahan kebutuhan atau perbaikan. Model dan proses harus mudah diperbarui tanpa memerlukan rekonstruksi total.
- **Rapidity (Kecepatan):** Bagaimana tingkat kecepatan proses untuk dapat memenuhi seluruh spesifikasi yang diminta secara lengkap. Proses yang baik harus efisien dalam hal waktu dan sumber daya tanpa mengorbankan kualitas.

2 Lanskap Metodologi Analisis

Secara garis besar, pendekatan pemodelan analisis terbagi menjadi dua mazhab besar: **Analisis Terstruktur** (klasik) dan **Analisis Berorientasi Objek** (modern). Analisis Terstruktur menggunakan pendekatan *top-down* yang memecah sistem menjadi komponen-komponen data dan proses yang lebih kecil. Sebaliknya, Analisis Berorientasi Objek menggabungkan data dan perilaku ke dalam entitas yang disebut objek.

Pada modul ini, kita akan berfokus pada Metodologi Berorientasi Objek (*Object-Oriented*). Sebelum standar UML diresmikan pada tahun 1997, ada banyak pakar yang menciptakan metode *Object-Oriented* (OO) mereka sendiri dengan notasi dan terminologi yang berbeda-beda. Berikut adalah 6 metode OO paling bersejarah yang akhirnya melebur menjadi UML:

1. Metode Booch (Object Oriented Design)

Metode ini dikembangkan oleh Grady Booch dan membagi proses analisis dan desain ke dalam 4 tahapan iteratif (berulang):

- (1) Identifikasi kelas dan objek dari domain masalah
- (2) Identifikasi semantik dan hubungan antar objek
- (3) Perincian *interface* (antarmuka) dan implementasi
- (4) Dokumentasi dan verifikasi

Metode ini terkenal dengan diagram object-oriented yang detail dan notation yang comprehensive.

2. Metode Rumbaugh (Object Modelling Technique - OMT)

Berdasarkan pada analisis terstruktur dan pemodelan *Entity Relationship* (ER). Memiliki 3 tahap utama:

- (1) Analisis Object Model, Dynamic Model, dan Functional Model
- (2) Desain Sistem dan desain Object-oriented
- (3) Implementasi dan pengujian

Keunggulannya adalah memiliki notasi yang mendukung semua konsep OO dan mudah diterapkan oleh tim yang sudah terbiasa dengan ER diagram.

3. Metode Jacobson (Object Oriented Software Engineering - OOSE)

Metode inilah yang menjadi cikal bakal (pioneer) konsep *Use-Case* yang sangat populer hingga saat ini. Tahapannya meliputi:

- (1) Model *requirement* dan analisis menggunakan use-case
- (2) Desain dan implementasi architecture
- (3) Model pengujian (test model)

Keunggulannya adalah notasinya sangat sederhana, mudah dipelajari, dan sangat efektif untuk komunikasi dengan stakeholder non-teknis.

4. Metode Coad dan Yourdon

Berfokus pada pemodelan OO dan ER dengan 4 komponen perancangan:

- (1) Problem domain (domain masalah)

- (2) Human interaction (interaksi manusia / UI)
- (3) Data management (manajemen data / persistence)
- (4) Task management (manajemen tugas / workflow)

Metode ini sangat aplikatif untuk sistem bisnis yang kompleks.

5. Metode Wirfs-Brock (RDD / CRC - Responsibility Driven Design)

Metode *Responsibility Driven Design* (RDD) ini sangat berguna untuk memunculkan ide di tahap awal analisis. Metode ini menggunakan kartu CRC (*Class Responsibility Collaboration*) untuk mengidentifikasi:

- (1) Kelas dan tanggung jawab (responsibility) masing-masing
- (2) Kolaborasi antar kelas
- (3) Hierarki pewarisan kelas

Pendekatan ini sangat interaktif dan sering digunakan dalam workshop analisis kelompok.

6. Metode Shlaer-Mellor (OOA/D Tradisional)

Menghasilkan tiga jenis model yang saling melengkapi:

- (1) Information model (diagram class/entity)
- (2) State model (diagram state/transisi state)
- (3) Process model (diagram aksi dan aktivitas)

Metode ini sangat mudah dikonversi dari metode struktural lama ke OO, sehingga cocok untuk organisasi yang sedang melakukan transisi dari structured programming ke object-oriented.

3 Pengenalan Unified Modeling Language (UML)

Karena dulu banyak pakar yang membuat standar diagramnya sendiri-sendiri, masing-masing dengan kelebihan dan kelemahan tersendiri, akhirnya pada pertengahan 1990-an disepakatilah sebuah standar global yang menggabungkan praktik terbaik dari semua metode di atas. Standar tersebut bernama **Unified Modeling Language (UML)**, yang diresmikan oleh *Object Management Group (OMG)* pada tahun 1997.

UML adalah kumpulan notasi yang lengkap untuk membuat visualisasi model suatu sistem. Notasi-notasi ini bersifat grafis dan memiliki semantik yang jelas, sehingga dapat dipahami oleh berbagai stakeholder. Tujuan utama diciptakannya UML adalah:

1. Memberikan bahasa pemodelan visual yang ekspresif, siap pakai, dan mudah dimengerti secara umum agar pengembang dapat saling menukar model dan berkomunikasi dengan efektif.
2. Memberikan standar pemodelan yang independen (bebas dari ikatan bahasa pemrograman tertentu, platform tertentu, atau proses rekayasa tertentu), sehingga UML dapat diterapkan di berbagai konteks dan lingkungan.
3. Menyatukan *best-practices* (praktik-praktik terbaik) yang selama ini digunakan dalam dunia pemodelan sistem, menciptakan satu bahasa bersama untuk industri software engineering.

3.1 Kategori Diagram dalam UML

Ada beberapa jenis diagram utama dalam UML yang dapat dikelompokkan berdasarkan fungsinya:

- **Diagram Struktur (Structural Diagrams):** Menggambarkan elemen-elemen statis dalam sistem seperti kelas, objek, komponen, dan hubungan antar mereka. Termasuk: Class Diagram, Object Diagram, Component Diagram, Deployment Diagram, Package Diagram.
- **Diagram Perilaku (Behavioral Diagrams):** Menggambarkan aspek dinamis atau perilaku sistem, bagaimana sistem berinteraksi dan bereaksi terhadap events. Termasuk: Use Case Diagram, Sequence Diagram, Activity Diagram, State Machine Diagram, Communication Diagram, Interaction Overview Diagram.

Mari kita bahas beberapa diagram utama secara mendetail.

4 1. Use Case Diagram

Use Case Diagram merupakan sebuah diagram yang bertujuan untuk menggambarkan **fungsionalitas** (feature atau capability) yang diharapkan dari sebuah sistem. Diagram ini fokus mendeskripsikan interaksi atau hubungan antara *Aktor* (pengguna/pihak luar) dengan *Sistem* itu sendiri melalui sebuah skenario cerita penggunaan (use case).

Use Case Diagram sangat bermanfaat pada tahap awal analisis kebutuhan (*requirements analysis*) karena:

- Membantu mengidentifikasi semua pengguna sistem (aktor) dan kebutuhan mereka
- Memberikan gambaran high-level tentang apa yang bisa dilakukan sistem
- Memfasilitasi komunikasi antara developer dan stakeholder
- Menjadi dasar untuk pembuatan test cases dan dokumentasi kebutuhan

Ada 6 elemen/komponen pembentuk Use Case Diagram:

1. System (Sistem)

Menyatakan batasan (*boundary*) ruang lingkup sistem. Digambarkan dengan kotak segi empat yang membatasi semua use case di dalamnya. Semua aktor harus berada di luar kotak ini. (Catatan: Seringkali kotak batas sistem ini disembunyikan/tidak digambar karena dianggap sudah jelas dari konteks).

2. Actor (Aktor)

Aktor adalah entitas di luar sistem yang akan menggunakan atau berinteraksi dengan sistem tersebut. Aktor tidak selalu manusia; ia bisa berupa *device* (perangkat keras seperti sensor, printer) atau bahkan sistem/server lain (seperti database server, payment gateway). Cara mudah mencari aktor adalah dengan bertanya: “Siapa/apa yang akan menggunakan sistem ini?” dan “Apakah sistem ini memberikan nilai/manfaat bagi dia?” Setiap aktor dalam diagram harus mempunyai setidaknya satu use case.

3. Use Case (Fungsionalitas)

Mengidentifikasi fitur atau *goal* (tujuan) utama dari sistem yang dicapai melalui interaksi dengan aktor. Digambarkan dengan bentuk Elips oval dengan nama aktivitas di dalamnya. Setiap use case pasti memiliki pemicu (*trigger*) yang memulainya. *Trigger* bisa bersifat **eksternal** (misal: user menekan tombol pesan) atau **temporal/waktu** (misal: alarm berbunyi saat jadwal jatuh tempo).

4. Association (Asosiasi)

Garis penghubung yang mengidentifikasi interaksi antara aktor dengan use case tertentu. Bisa berupa garis lurus tanpa panah (interaksi dua arah) atau dengan panah (komunikasi satu arah). Association mewakili partisipasi dari aktor dalam use case tersebut.

5. Dependency (Ketergantungan)

Garis putus-putus berpanah yang menunjukkan hubungan ketergantungan antar dua Use Case. Terdiri dari 2 jenis:

- **«include» (Inclusion)**: Use case yang satu WAJIB memanggil use case yang lain agar bisa selesai dengan sempurna. Hubungan ini bersifat unconditional dan mandatory. (Arah panah menuju ke use case yang dipanggil/included).
- **«extend» (Extension)**: Pemanggilan use case tambahan HANYA JIKA kondisi tertentu terpenuhi (opsional). Hubungan ini bersifat conditional dan optional. (Arah panah berlawanan, menuju ke use case pemanggil/yang diperluas).

6. Generalization (Generalisasi)

Mendefinisikan relasi pewarisan sifat (*inheritance*) antar use case atau antar aktor. Digambarkan dengan garis bermata panah kosong (segitiga putih) yang mengarah dari sisi *Child* (anak) ke sisi *Parent* (induk). Ini menunjukkan bahwa aktor child mewarisi semua karakteristik dari parent.

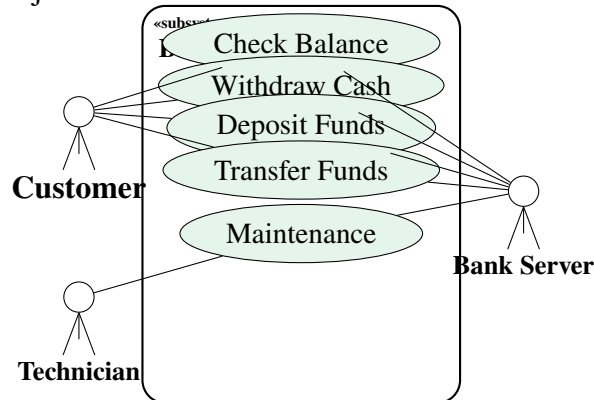


Figure 1: Contoh Use Case Diagram Sistem Bank ATM - Menunjukkan aktor (Customer, Technician, Bank Server) dan use case (Check Balance, Withdraw Cash, dll) dengan asosiasi dan batasan sistem.

5 2. Activity Diagram

Jika Use Case fokus pada “**Apa**” yang bisa dilakukan sistem, **Activity Diagram** berfokus pada “**Bagaimana**” alur kerja (*workflow*) dan proses bisnis sistem tersebut secara rinci. Activity diagram menampilkan urutan (*sequence*) dari aktivitas-aktivitas, keputusan-keputusan (*decision points*), dan sinkronisasi (*synchronization*) antar aktivitas.

Activity diagram menggambarkan berbagai alur aktivitas dalam sistem:

- Bagaimana masing-masing alur berawal (*starting point*)
- Pengambilan keputusan (*decision*) yang mungkin terjadi dan kondisi-kondisinya
- Bagaimana alur tersebut berakhir (*ending point*)
- Aktivitas yang berjalan paralel atau bersamaan (*concurrent activities*)
- Objek atau data yang mengalir dari satu aktivitas ke aktivitas lain

Perlu dipahami bahwa Activity Diagram ini tidak menggambarkan *behaviour* (perilaku) eksak internal antar subsistem, melainkan lebih ke gambaran proses bisnis level atas secara umum. Oleh karena itu, Activity Diagram sangat cocok untuk komunikasi dengan non-technical stakeholder seperti business analyst atau user.

5.1 Simbol-Simbol Activity Diagram

Terdapat beberapa notasi simbol standar yang harus digunakan dalam activity diagram:

5.2 Swimlane dalam Activity Diagram

Fitur penting lainnya di Activity Diagram adalah **Swimlane** (juga disebut *partition*). *Swimlane* ini membagi area diagram ke dalam beberapa partisi kolom (mirip jalur renang dalam kolam renang) yang mewakili entitas, aktor, atau departemen mana yang sedang bertanggung jawab melakukan suatu aktivitas pada saat itu. Dengan swimlane, kita dapat dengan jelas melihat siapa responsible untuk apa, sehingga sangat berguna untuk menunjukkan ownership dan accountability dalam proses bisnis. Swimlane dapat diatur secara horizontal (*vertical partitions* untuk aktor) atau vertikal (*horizontal partitions* untuk organizational units).

Nama Simbol	Keterangan / Fungsi
Action/Activity	Digambarkan dengan kotak dengan sudut yang membulat (<i>rounded rectangle</i>). Merupakan state yang mencerminkan eksekusi dari suatu aksi atau pekerjaan konkret. Contoh: “Proses Pembayaran”, “Update Database”.
Initial Node	Titik hitam bulat penuh (filled circle). Menandakan bagaimana aktivitas sistem bermula atau dimulai (Start point). Setiap diagram harus memiliki exactly satu initial node.
Final Node	Titik hitam dengan lingkaran luar (target symbol). Menandakan bagaimana aktivitas sistem diakhiri secara keseluruhan (End point). Diagram dapat memiliki multiple final nodes untuk berbagai skenario akhir.
Decision	Bentuk belah ketupat (<i>diamond</i>). Digunakan untuk menggambarkan suatu keputusan atau tindakan bercabang yang harus diambil pada kondisi tertentu (IF-ELSE logic). Edge yang keluar dari decision harus memiliki label kondisi (guard condition).
Merge	Bentuk belah ketupat dengan multiple edge input tapi single output. Digunakan untuk menggabungkan beberapa alur menjadi satu setelah decision point. Berbeda dengan decision yang membagi alur.
Fork/Join Bar	Garis horizontal tebal. Fork memisahkan satu alur menjadi beberapa alur paralel yang berjalan bersamaan. Join menggabungkan beberapa alur paralel menjadi satu dan menunggu semua alur selesai.
Control Flow	Garis panah yang menghubungkan satu simbol aktivitas ke simbol lainnya sesuai urutan eksekusi. Label pada garis dapat menunjukkan kondisi atau event.

6 3. Sequence Diagram

Sequence Diagram adalah diagram yang sangat teknis dan detail. Diagram ini menggambarkan interaksi antara objek-objek (atau komponen-komponen) di dalam dan di sekitar sistem. Interaksi ini mencakup pengguna (actor), antarmuka pengguna (UI), pengontrol (controller), logika bisnis (business logic), dan database. Sequence diagram memperlihatkan urutan waktu dari setiap message (pesan) yang dikirim dan diterima.

Karakteristik utama Sequence Diagram adalah:

- Menggambarkan interaksi berupa **pesan (message)** yang dikirim dan diterima antar objek. Pesan dapat berupa method call, return value, atau event.
- Interaksi ini dipetakan secara terstruktur **terhadap waktu**. Waktu mengalir dari atas ke bawah, sehingga event yang lebih atas terjadi lebih dulu.
- Terdiri atas dua dimensi utama:

- (1) **Dimensi Vertikal** yang mewakili aliran waktu (semakin ke bawah = semakin akhir dalam timeline)
- (2) **Dimensi Horizontal** yang mewakili objek-objek yang sedang saling berkomunikasi

Sequence diagram sangat berguna karena:

- Menunjukkan detail alur eksekusi dari setiap use case
- Membantu tim developer memahami urutan method call dan data flow
- Mengidentifikasi potential issues seperti deadlock atau race condition
- Menjadi blueprint untuk implementasi kode

6.1 Simbol-Simbol Sequence Diagram

Elemen pembentuk diagram ini antara lain:

Nama Simbol	Keterangan / Fungsi
Actor/Object Box	Kotak header di bagian atas diagram yang merepresentasikan aktor atau objek. Dituliskan dengan format <code>objectName : ClassName</code> .
LifeLine	Garis vertikal putus-putus yang membentang ke bawah dari setiap object box. Menggambarkan eksistensi atau periode hidup dari objek, entitas, atau antarmuka tersebut selama waktu tertentu.
Message	Garis panah horizontal penuh (solid arrow). Spesifikasi komunikasi antar objek yang memuat aksi, pemanggilan fungsi (<i>method</i>), atau perintah yang dieksekusi oleh objek tujuan. Dapat memiliki parameter dan return type.
Return Message	Garis panah horizontal putus-putus (dashed arrow). Menandakan nilai kembalian atau respon balasan dari objek yang dipanggil kembali ke objek pemanggil. Dapat disertai dengan nilai return.
Activation Box	Kotak vertikal tipis pada lifeline yang menunjukkan periode waktu ketika objek sedang aktif (melakukan pekerjaan atau menunggu response). Dimulai saat method dipanggil dan berakhir saat method selesai/return.
Alt/Opt/Loop Frame	Kotak interaksi besar yang membungkus beberapa message untuk menunjukkan conditional logic (<i>Alt</i>), optional behavior (<i>Opt</i>), atau repeated behavior (<i>Loop</i>).

Dengan Sequence Diagram, tim *developer* bisa dengan sangat presisi melihat:

- Urutan *method* apa saja yang harus di-*coding* dan dalam urutan apa
- Bagaimana pertukaran data antar kelas (objek) terjadi

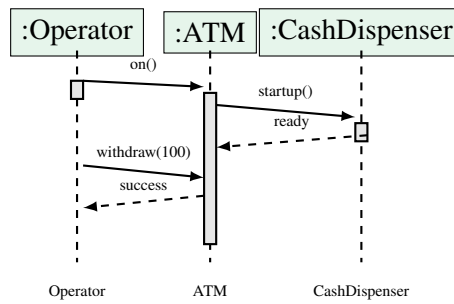


Figure 2: Contoh Potongan Sequence Diagram Sistem ATM - Menunjukkan komunikasi antara Operator, ATM, dan CashDispenser dengan message flow (solid arrow) dan return value (dashed arrow).

- Parameter yang dibutuhkan dan nilai return yang diharapkan
- Sinkronisasi atau koordinasi antar objek yang kompleks

Sequence diagram menjadi blueprint yang sangat akurat untuk fase implementasi dan integration testing.